

Week 3 Module

DELTA
Date / /
Pg No.

Merge Sort

Merge Sort is a highly efficient, comparison based, divide-and-conquer sorting algorithm.

Merge Sort works by repeatedly dividing the array into smaller subarrays until each subarray contains a single element.

These smaller subarrays are then merged in a sorted manner to produce the final sorted array.

Always $O(N \log N)$ in all cases (Best, Average, worst)

$O(N)$ due to the auxiliary array used during the merging phase

Week 3 Module:

Date: 29/06/2026

Topic 1: Merge Sort

Question 1 (Easy) – Rank Students by Marks

A school has recently conducted a scholarship examination for all students of Class 12. The examination department has collected the marks of all students in an array. Due to the large number of participants, manually arranging the marks in ascending order has become difficult.

Your task is to write a program that takes the marks of N students and sorts them in ascending order using the Merge Sort algorithm.

Merge Sort works by repeatedly dividing the array into smaller subarrays until each subarray contains a single element. These smaller subarrays are then merged in a sorted manner to produce the final sorted array.

The school administration wants the marks to be displayed in increasing order so they can easily identify the lowest and highest scoring students and generate performance reports.

Input

- First line contains an integer N, representing the number of students.
- Second line contains N space-separated integers, representing the marks of the students.

Output

- Print the marks in ascending order.

Constraints

- $1 \leq N \leq 10^5$
- $0 \leq \text{Marks} \leq 100$

Example

Input:

6
78 45 92 34 67 81

Output:

34 45 67 78 81 92

Q.1) Rank Students by Marks

```
#include <stdio.h>
// function to merge two halves of an array
void merge(int arr[], int left, int mid,
           int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    // Temporary arrays
    int L[n1], R[n2];

    // copy data to temporary arrays L[] and R[]
    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;

    // merge the temporary arrays back into
    // arr[left, right)
    while (i < n1 & j < n2) {
        if (L[i] <= R[j])
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }
```


3

// Copy the remaining elements of L[], if there
are any
while (i < n1) {
arr[k] = L[i];
i++;
k++;
}

// Copy the remaining elements of R[], if there
are any
while (j < n2) {
arr[k] = R[j];
j++;
k++;
}

// Main function to perform merge sort
void mergesort(int arr[], int left, int right) {
if (left < right) {

// Calculate mid point to avoid overflow
int mid = left + (right - left) / 2;

// Sort first and second halves
mergesort(arr, left, mid);
mergesort(arr, mid + 1, right);

DELTA
Date / /
Pg No.
// merge the sorted halves
merge(arr, left, mid, right);
3 3

int main() {
int N;
scanf("%d", &N); // Number of students

int marks[N];
for (int i=0; i<N; i++) {
scanf("%d", &marks[i]);
}

mergesort(marks, 0, N-1);

for (int i=0; i<N; i++) {
printf("%d", marks[i]);
}

printf("\n");
return 0;
}

I/p:- 6 78 45 92 34 67 81

O/p:- 34 45 67 78 81 92



main.c

```
1  #include <stdio.h>
2
3  // Function to merge two halves of an array
4  void merge(int arr[], int left, int mid, int right) {
5      int n1 = mid - left + 1;
6      int n2 = right - mid;
7
8      // Temporary arrays
9      int L[n1], R[n2];
10
11     // Copy data to temporary arrays L[] and R[]
12     for (int i = 0; i < n1; i++)
13         L[i] = arr[left + i];
14     for (int j = 0; j < n2; j++)
15         R[j] = arr[mid + 1 + j];
16
17     int i = 0, j = 0, k = left;
18
19     // Merge the temporary arrays back into arr[left..right]
20     while (i < n1 && j < n2) {
21         if (L[i] <= R[j]) {
22             arr[k] = L[i];
23             i++;
24         } else {
25             arr[k] = R[j];
26             j++;
27         }
28         k++;
29     }
30
31     // Copy the remaining elements of L[], if there are any
32     while (i < n1) {
33         arr[k] = L[i];
34         i++;
35         k++;
36     }
```



main.c

```
27     }
28     k++;
29 }
30
31 // Copy the remaining elements of L[], if there are any
32 while (i < n1) {
33     arr[k] = L[i];
34     i++;
35     k++;
36 }
37
38 // Copy the remaining elements of R[], if there are any
39 while (j < n2) {
40     arr[k] = R[j];
41     j++;
42     k++;
43 }
44 }
45
46 // Main function to perform Merge Sort
47 void mergeSort(int arr[], int left, int right) {
48     if (left < right) {
49         // Calculate mid point to avoid overflow
50         int mid = left + (right - left) / 2;
51
52         // Sort first and second halves
53         mergeSort(arr, left, mid);
54         mergeSort(arr, mid + 1, right);
55
56         // Merge the sorted halves
57         merge(arr, left, mid, right);
58     }
59 }
60
```



main.c

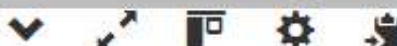
```
43     }
44 }
45
46 // Main function to perform Merge Sort
47 void mergeSort(int arr[], int left, int right) {
48     if (left < right) {
49         // Calculate mid point to avoid overflow
50         int mid = left + (right - left) / 2;
51
52         // Sort first and second halves
53         mergeSort(arr, left, mid);
54         mergeSort(arr, mid + 1, right);
55
56         // Merge the sorted halves
57         merge(arr, left, mid, right);
58     }
59 }
60
61 int main() {
62     int N;
63     scanf("%d", &N); // Number of students
64
65     int marks[N];
66     for(int i = 0; i < N; i++) {
67         scanf("%d", &marks[i]);
68     }
69
70     mergeSort(marks, 0, N - 1);
71
72     for(int i = 0; i < N; i++) {
73         printf("%d ", marks[i]);
74     }
75     printf("\n");
76
77     return 0;
78 }
```



```

52 // Sort first and second halves
53 mergeSort(arr, left, mid);
54 mergeSort(arr, mid + 1, right);
55
56 // Merge the sorted halves
57 merge(arr, left, mid, right);
58 }
59 }
60
61 int main() {
62     int N;
63     scanf("%d", &N); // Number of students
64
65     int marks[N];
66     for(int i = 0; i < N; i++) {
67         scanf("%d", &marks[i]);
68     }
69
70     mergeSort(marks, 0, N - 1);

```



6 78 45 92 34 67 81

34 45 67 78 81 92

...Program finished with exit code 0

Press ENTER to exit console.

Question 2 (Easy) – Organizing Product Prices

An e-commerce company stores product prices in the order they are added to the platform. Before publishing a seasonal sales report, the company wants all product prices to be arranged in ascending order.

Since the number of products can be very large, an efficient sorting technique is required. Your task is to implement Merge Sort to arrange the prices from the cheapest product to the most expensive product.

The program should divide the list into smaller parts, sort each part, and merge them back together while maintaining the correct order.

Input

- First line contains integer N.
- Second line contains N space-separated product prices.

Output

- Print the sorted prices in ascending order.

Constraints

- $1 \leq N \leq 100000$
- $1 \leq \text{Price} \leq 1000000$

Example

Input:

5
2500 1500 4000 3000 1000

Output:

1000 1500 2500 3000 4000

Q.2) Organizing Product prices

```
#include <stdio.h>
#include <stdlib.h>
```

```
void merge (long long arr[], int left, int mid,
            int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;
    long long L[n1], R[n2];
```

```
    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];
```

```
    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k++] = L[i++];
```

```
        } else {
            arr[k++] = R[j++];
        }
    }
```

```
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}
```

```
void mergeSort (long long arr[], int left,
                int right) {
```

```
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort (arr, left, mid);
```


DELTA
Date / /
Pg No

```

mergeSort(arr, mid + 1, right);
merge(arr, left, mid, right);
}

```

```

int main() {
int N;
scanf("%d", &N);

```

// Using dynamically allocated array
for large constraints
long long *prices = (long long *) malloc
(N * sizeof(long long));

```

for(int i = 0; i < N; i++) {
scanf("%lld", &prices[i]);
}

```

```

mergeSort(prices, 0, N - 1);

```

```

for(int i = 0; i < N; i++) {
printf("%lld", prices[i]);
}

```

```

printf("\n");
free(prices);
return 0;
}

```

I/P: - 5 2500 1500 4000 3000 1000

O/P: - 1000 1500 2500 3000 4000

main.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  void merge(long long arr[], int left, int mid, int right) {
5      int n1 = mid - left + 1;
6      int n2 = right - mid;
7      long long L[n1], R[n2];
8
9      for (int i = 0; i < n1; i++) L[i] = arr[left + i];
10     for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];
11
12     int i = 0, j = 0, k = left;
13     while (i < n1 && j < n2) {
14         if (L[i] <= R[j]) {
15             arr[k++] = L[i++];
16         } else {
17             arr[k++] = R[j++];
18         }
19     }
20     while (i < n1) arr[k++] = L[i++];
21     while (j < n2) arr[k++] = R[j++];
22 }
23
24 void mergeSort(long long arr[], int left, int right) {
25     if (left < right) {
26         int mid = left + (right - left) / 2;
27         mergeSort(arr, left, mid);
28         mergeSort(arr, mid + 1, right);
29         merge(arr, left, mid, right);
30     }
31 }
32
33 int main() {
34     int N;
35     scanf("%d", &N);
36 }
```

input

```

19     }
20     while (i < n1) arr[k++] = L[i++];
21     while (j < n2) arr[k++] = R[j++];
22 }
23
24 void mergeSort(long long arr[], int left, int right) {
25     if (left < right) {
26         int mid = left + (right - left) / 2;
27         mergeSort(arr, left, mid);
28         mergeSort(arr, mid + 1, right);
29         merge(arr, left, mid, right);
30     }
31 }
32
33 int main() {
34     int N;
35     scanf("%d", &N);
36
37     // Using dynamically allocated array for large constraints
38     long long *prices = (long long *)malloc(N * sizeof(long long));
39
40     for(int i = 0; i < N; i++) {
41         scanf("%lld", &prices[i]);
42     }
43
44     mergeSort(prices, 0, N - 1);
45
46     for(int i = 0; i < N; i++) {
47         printf("%lld ", prices[i]);
48     }
49     printf("\n");
50
51     free(prices);
52     return 0;
53 }

```



```

30     }
31 }
32
33 int main() {
34     int N;
35     scanf("%d", &N);
36
37     // Using dynamically allocated array for large constraints
38     long long *prices = (long long *)malloc(N * sizeof(long long));
39
40     for(int i = 0; i < N; i++) {
41         scanf("%lld", &prices[i]);
42     }
43
44     mergeSort(prices, 0, N - 1);
45

```



```

5 2500 1500 4000 3000 1000
1000 1500 2500 3000 4000

```

```

...Program finished with exit code 0
Press ENTER to exit console.

```

Question 3 (Medium) – Sort Employee Records by Salary

A multinational company has employee salary records stored in an array. Before generating annual reports, the Human Resources department wants the salary records arranged in ascending order.

The company expects the number of employees to grow significantly over time, so an efficient sorting algorithm must be used. You are required to implement Merge Sort and display the salaries in increasing order.

Additionally, after sorting, the company wants to know:

1. The lowest salary.
2. The highest salary.
3. The difference between the highest and lowest salary.

Input

- First line contains integer N.
- Second line contains N space-separated salaries.

Output

- First line: Sorted salaries.
- Second line: Lowest salary.
- Third line: Highest salary.
- Fourth line: Salary difference.

Constraints

- $1 \leq N \leq 100000$
- $1000 \leq \text{Salary} \leq 1000000$

Example

Input:

7

45000 25000 70000 50000 30000 90000 60000

Output:

25000 30000 45000 50000 60000 70000 90000

Lowest Salary: 25000

Highest Salary: 90000

Difference: 65000

Q.3) Sort Employee Records by salary

```
#include <stdio.h>
#include <stdlib.h>
```

```
void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1, n2 = right - mid;
    int L[n1], R[n2];
```

```
    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];
```

```
    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }
```

```
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}
```

```
void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}
```



```

int main() {
    int N;
    scanf("%d", &N);

    int *salaries = (int *) malloc (N * sizeof(int));
    for (int i = 0; i < N; i++) {
        scanf("%d", &salaries[i]);
    }
    mergesort(salaries, 0, N-1);

    // 1. Sorted Salaries
    for (int i = 0; i < N; i++) {
        printf("%d", salaries[i]);
    }
    printf("\n");

    // 2. Lowest, Highest and Difference
    printf("Lowest Salary: %d\n", salaries[0]);
    printf("Highest Salary: %d\n", salaries[N-1]);
    printf("Difference: %d\n", salaries[N-1] - salaries[0]);

    free(salaries);
    return 0;
}

```

I/P: - 7 45000 25000 70000 50000 30000 90000
60000

O/P: - 25000 30000 45000 50000 60000 70000 90000
Lowest salary: 25000 Highest salary: 90000
Difference: 65000



main.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  void merge(int arr[], int left, int mid, int right) {
5      int n1 = mid - left + 1, n2 = right - mid;
6      int L[n1], R[n2];
7
8      for (int i = 0; i < n1; i++) L[i] = arr[left + i];
9      for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];
10
11     int i = 0, j = 0, k = left;
12     while (i < n1 && j < n2) {
13         if (L[i] <= R[j]) arr[k++] = L[i++];
14         else arr[k++] = R[j++];
15     }
16     while (i < n1) arr[k++] = L[i++];
17     while (j < n2) arr[k++] = R[j++];
18 }
19
20 void mergeSort(int arr[], int left, int right) {
21     if (left < right) {
22         int mid = left + (right - left) / 2;
23         mergeSort(arr, left, mid);
24         mergeSort(arr, mid + 1, right);
25         merge(arr, left, mid, right);
26     }
27 }
28
29 int main() {
30     int N;
31     scanf("%d", &N);
32
33     int *salaries = (int *)malloc(N * sizeof(int));
34     for(int i = 0; i < N; i++) {
35         scanf("%d", &salaries[i]);
36     }
```



```

18 }
19
20 void mergeSort(int arr[], int left, int right) {
21     if (left < right) {
22         int mid = left + (right - left) / 2;
23         mergeSort(arr, left, mid);
24         mergeSort(arr, mid + 1, right);
25         merge(arr, left, mid, right);
26     }
27 }
28
29 int main() {
30     int N;
31     scanf("%d", &N);
32
33     int *salaries = (int *)malloc(N * sizeof(int));
34     for(int i = 0; i < N; i++) {
35         scanf("%d", &salaries[i]);
36     }
37
38     mergeSort(salaries, 0, N - 1);
39
40     // 1. Sorted Salaries
41     for(int i = 0; i < N; i++) {
42         printf("%d ", salaries[i]);
43     }
44     printf("\n");
45
46     // 2. Lowest, Highest and Difference
47     printf("Lowest Salary: %d\n", salaries[0]);
48     printf("Highest Salary: %d\n", salaries[N - 1]);
49     printf("Difference: %d\n", salaries[N - 1] - salaries[0]);
50
51     free(salaries);
52     return 0;
53 }

```



```

27 }
28
29 int main() {
30     int N;
31     scanf("%d", &N);
32
33     int *salaries = (int *)malloc(N * sizeof(int));
34     for(int i = 0; i < N; i++) {
35         scanf("%d", &salaries[i]);
36     }
37
38     mergeSort(salaries, 0, N - 1);
39
40     // 1. Sorted Salaries
41     for(int i = 0; i < N; i++) {
42         printf("%d ", salaries[i]);
43     }
44     printf("\n");
45

```



25000 30000 45000 50000 60000 70000 90000

Lowest Salary: 25000

Highest Salary: 90000

Difference: 65000

...Program finished with exit code 0

Press ENTER to exit console.

Quick sort

Quick sort is a fast, highly optimized, divide and conquer sorting algorithm.

It works by picking a 'pivot' element from the array and partitioning the other elements into two sub-arrays according to whether they are less than or greater than the pivot.

Average case is $O(N \log N)$. worst case is $O(N^2)$ (occurs when the array is already sorted, and the pivot chosen is always the extreme element)

$O(\log N)$ due to recursive stack space.
Quick Sort is an in-place sorting algorithm.

Topic 2: Quick Sort

Question 1 (Easy) – Race Timing Analysis

A sports academy records the completion times (in seconds) of runners participating in a race. The coach wants to arrange the timings from the fastest runner to the slowest runner.

To process large datasets efficiently, the academy has decided to use the Quick Sort algorithm. Your task is to sort the race timings in ascending order.

Input

- First line contains integer N.
- Second line contains N space-separated timings.

Output

- Print the timings in ascending order.

Constraints

- $1 \leq N \leq 100000$
- $1 \leq \text{Time} \leq 10000$

Example

Input:

6
55 49 62 58 45 51

Output:

45 49 51 55 58 62

DELTA
Date / /
Pg No.

Q.1) Race Timing Analysis

```
#include <stdio.h>
```

```
// function to swap two elements
```

```
void swap(int *a, int *b) {
```

```
    int t = *a;
```

```
    *a = *b;
```

```
    *b = t;
```

```
}
```

```
// partition function for Quick Sort
```

```
int partition(int arr[], int low, int high) {
```

```
    int pivot = arr[high]; // choosing the last element  
                           as pivot
```

```
    int i = (low - 1); // index of smallest element
```

```
    for (int j = low; j <= high - 1; j++) {
```

```
        // If current element is smaller than the  
        pivot
```

```
        if (arr[j] < pivot) {
```

```
            i++;
```

```
            swap(&arr[i], &arr[j]);
```

```
        }
```

```
        swap(&arr[i+1], &arr[high]);
```

```
        return (i+1);
```

```
}
```

```
// main Quick Sort function
```

```
void quickSort(int arr[], int low, int high) {
```

```
    if (low < high) {
```

```
// partition index
int pi = partition(arr, low, high);
```

```
// separately sort elements before and after partition
quickSort(arr, low, pi-1);
quickSort(arr, pi+1, high);
}
```

```
int main()
{
    int N;
    scanf("%d", &N);
```

```
int timings[N];
for(int i = 0; i < N; i++) {
    scanf("%d", &timings[i]);
}
quickSort(timings, 0, N-1);
for(int i = 0; i < N; i++) {
    printf("%d", timings[i]);
}
printf("\n");
return 0;
}
```

I/p:- 6 55 49 62 58 45 51

O/p:- 45 49 51 55 58 62



main.c

```
1  #include <stdio.h>
2
3  // Function to swap two elements
4  void swap(int* a, int* b) {
5      int t = *a;
6      *a = *b;
7      *b = t;
8  }
9
10 // Partition function for Quick Sort
11 int partition(int arr[], int low, int high) {
12     int pivot = arr[high]; // Choosing the last element as pivot
13     int i = (low - 1); // Index of smaller element
14
15     for (int j = low; j <= high - 1; j++) {
16         // If current element is smaller than the pivot
17         if (arr[j] < pivot) {
18             i++;
19             swap(&arr[i], &arr[j]);
20         }
21     }
22     swap(&arr[i + 1], &arr[high]);
23     return (i + 1);
24 }
25
26 // Main Quick Sort function
27 void quickSort(int arr[], int low, int high) {
28     if (low < high) {
29         // Partition index
30         int pi = partition(arr, low, high);
31
32         // Separately sort elements before and after partition
33         quickSort(arr, low, pi - 1);
34         quickSort(arr, pi + 1, high);
35     }
36 }
```

input


```

20     }
21 }
22 swap(&arr[i + 1], &arr[high]);
23 return (i + 1);
24 }
25
26 // Main Quick Sort function
27 void quickSort(int arr[], int low, int high) {
28     if (low < high) {
29         // Partition index
30         int pi = partition(arr, low, high);
31
32         // Separately sort elements before and after partition
33         quickSort(arr, low, pi - 1);
34         quickSort(arr, pi + 1, high);
35     }
36 }
37
38 int main() {
39     int N;
40     scanf("%d", &N);
41
42     int timings[N];
43     for(int i = 0; i < N; i++) {
44         scanf("%d", &timings[i]);
45     }
46
47     quickSort(timings, 0, N - 1);
48
49     for(int i = 0; i < N; i++) {
50         printf("%d ", timings[i]);
51     }
52     printf("\n");
53
54     return 0;
55 }

```

```

26 // Main Quick Sort function
27 void quickSort(int arr[], int low, int high) {
28     if (low < high) {
29         // Partition index
30         int pi = partition(arr, low, high);
31
32         // Separately sort elements before and after partition
33         quickSort(arr, low, pi - 1);
34         quickSort(arr, pi + 1, high);
35     }
36 }
37
38 int main() {
39     int N;
40     scanf("%d", &N);
41
42     int timings[N];
43     for(int i = 0; i < N; i++) {
44         scanf("%d", &timings[i]);
45     }
46
47     quickSort(timings, 0, N - 1);

```



input

```

6 55 49 62 58 45 51
45 49 51 55 58 62

```

```

...Program finished with exit code 0
Press ENTER to exit console.

```

Question 2 (Easy) – Organizing Library Book IDs

A library stores books using unique identification numbers. Due to years of additions and removals, the IDs are no longer arranged properly.

The librarian wants all IDs sorted in ascending order before migrating the data to a new management system. Implement Quick Sort to arrange the IDs efficiently.

Input

- First line contains integer N.
- Second line contains N space-separated book IDs.

Output

- Print the sorted IDs.

Constraints

- $1 \leq N \leq 100000$
- $1 \leq ID \leq 10^9$

Example

Input:

5
1025 1001 1050 1010 1005

Output:

1001 1005 1010 1025 1050

Q.2) Organizing Library Book IDs

```
#include <stdio.h>
#include <stdlib.h>
```

```
void swap (long long *a, long long *b) {
    long long t = *a;
    *a = *b;
    *b = t;
}
```

```
int partition (long long arr[], int low, int high) {
    long long pivot = arr[high];
    int i = (low-1);
```

```
    for (int j = low; j <= high-1; j++) {
        if (arr[j] < pivot) {
            i++;
            swap (&arr[i], &arr[j]);
        }
    }
```

```
    swap (&arr[i+1], &arr[high]);
    return (i+1);
}
```

```
void quicksort (long long arr[], int low, int high) {
```

```
    if (low < high) {
        int pi = partition (arr, low, high);
        quicksort (arr, low, pi-1);
        quicksort (arr, pi+1, high);
    }
}
```

int main() {
 int N;
 scanf("%d", &N);

// Book IDs can be up to 10^9 , using long long for safety

long long *bookIDs = (long long *) malloc
 (N * sizeof(long long));
 for(int i=0; i<N; i++) {
 scanf("%lld", &bookIDs[i]);
 }

quickSort(bookIDs, 0, N-1);

for(int i=0; i<N; i++) {
 printf("%lld", bookIDs[i]);
 }
 printf("\n");
 free(bookIDs);
 return 0;
 }

I/P:- 5 1025 1001 1000 1010 1005

O/P:- 1001 1005 1010 1025 1000



main.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  void swap(long long* a, long long* b) {
5      long long t = *a;
6      *a = *b;
7      *b = t;
8  }
9
10 int partition(long long arr[], int low, int high) {
11     long long pivot = arr[high];
12     int i = (low - 1);
13
14     for (int j = low; j <= high - 1; j++) {
15         if (arr[j] < pivot) {
16             i++;
17             swap(&arr[i], &arr[j]);
18         }
19     }
20     swap(&arr[i + 1], &arr[high]);
21     return (i + 1);
22 }
23
24 void quickSort(long long arr[], int low, int high) {
25     if (low < high) {
26         int pi = partition(arr, low, high);
27         quickSort(arr, low, pi - 1);
28         quickSort(arr, pi + 1, high);
29     }
30 }
31
32 int main() {
33     int N;
34     scanf("%d", &N);
35
36     // Book IDs can be up to 10^9, using long long for safety
```



```

17         swap(&arr[i], &arr[j]);
18     }
19 }
20 swap(&arr[i + 1], &arr[high]);
21 return (i + 1);
22 }
23
24 void quickSort(long long arr[], int low, int high) {
25     if (low < high) {
26         int pi = partition(arr, low, high);
27         quickSort(arr, low, pi - 1);
28         quickSort(arr, pi + 1, high);
29     }
30 }
31
32 int main() {
33     int N;
34     scanf("%d", &N);
35
36     // Book IDs can be up to 10^9, using Long Long for safety
37     long long *bookIDs = (long long *)malloc(N * sizeof(long long));
38     for(int i = 0; i < N; i++) {
39         scanf("%lld", &bookIDs[i]);
40     }
41
42     quickSort(bookIDs, 0, N - 1);
43
44     for(int i = 0; i < N; i++) {
45         printf("%lld ", bookIDs[i]);
46     }
47     printf("\n");
48
49     free(bookIDs);
50     return 0;
51 }

```

```

23
24 void quickSort(long long arr[], int low, int high) {
25     if (low < high) {
26         int pi = partition(arr, low, high);
27         quickSort(arr, low, pi - 1);
28         quickSort(arr, pi + 1, high);
29     }
30 }
31
32 int main() {
33     int N;
34     scanf("%d", &N);
35
36     // Book IDs can be up to 10^9, using long long for safety
37     long long *bookIDs = (long long *)malloc(N * sizeof(long long));
38     for(int i = 0; i < N; i++) {
39         scanf("%lld", &bookIDs[i]);
40     }
41
42     quickSort(bookIDs, 0, N - 1);
43

```

input

```

5 1025 1001 1050 1010 1005
1001 1005 1010 1025 1050

```

```

...Program finished with exit code 0
Press ENTER to exit console.

```

Question 3 (Medium) – Online Store Sales Ranking

An online marketplace tracks the daily sales made by different sellers. At the end of each week, management wants to identify the best-performing sellers.

You are given the total sales made by N sellers. Using Quick Sort, arrange the sales figures in descending order.

After sorting, display:

1. Sorted sales figures.
2. Top 3 highest sales values.
3. Average of the top 3 sales values.

Input

- First line contains integer N .
- Second line contains N space-separated sales values.

Output

- Sorted sales figures in descending order.
- Top 3 sales values.
- Average of top 3 sales.

Constraints

- $3 \leq N \leq 100000$
- $1 \leq \text{Sales} \leq 1000000$

Example

Input:

6
1200 5000 3000 2500 7000 4500

Output:

7000 5000 4500 3000 2500 1200
Top 3: 7000 5000 4500
Average: 5500

3) Online store sales Ranking

```
#include <stdio.h>
#include <stdlib.h>
```

```
void swap(long long *a, long long *b) {
    long long t = *a;
    *a = *b;
    *b = t;
}
```

// Modified partition function for Descending Order

```
int partitionDescending(long long arr[], int low, int high) {
    long long pivot = arr[high];
    int i = (low - 1);
```

```
    for (int j = low; j <= high - 1; j++) {
        // change logic to greater than for descending
        if (arr[j] > pivot) {
```

```
            i++;
            swap(&arr[i], &arr[j]);
        }
```

```
    }
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}
```

```
void quickSortDescending(long long arr[], int low, int high) {
```

```
    if (low < high) {
```

DELTA
Date / /
Pg No.

```
int pi = partitionDescending(arr, low, high);  
quickSortDescending(arr, low, pi-1);  
quickSortDescending(arr, pi+1, high);  
}
```

```
int main ()  
{  
    int N;  
    scanf("%d", &N);
```

```
    long long * sales = (long long *) malloc(N *  
        sizeof(long long));  
    for (int i=0; i<N; i++) {  
        scanf("%lld", &sales[i]);  
    }
```

```
    // sort in Descending order  
    quickSortDescending(sales, 0, N-1);
```

```
    // 1. Print sorted sales  
    for (int i=0; i<N; i++) {  
        printf("%lld", sales[i]);  
    }
```

```
    printf("\n");
```

```
    // 2. Print Top 3  
    printf("Top 3: ");  
    for (int i=0; i<3 && i<N; i++) {  
        printf("%lld", sales[i]);  
    }
```

```
    printf("\n");
```


// 3. Print Average of top 3

```
long long sum = 0;
int count = (N > 3) ? 3 : N;
for (int i = 0; i < count; i++) {
    sum += sales[i];
}
printf("Average: %.1ld\n", sum/count);
free(sales);
return 0;
```

I/P:-

6 1200 5000 3000 2000 7000 4500

O/P:-

7000 5000 4500 3000 2000 1200

Top 3: 7000 5000 4500

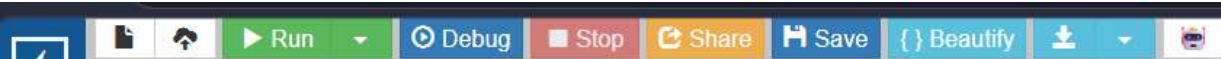
Average: 5500



Run Debug Stop Share Save {} Beautify

main.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 void swap(long long* a, long long* b) {
5     long long t = *a;
6     *a = *b;
7     *b = t;
8 }
9
10 // Modified Partition function for Descending Order
11 int partitionDescending(long long arr[], int low, int high) {
12     long long pivot = arr[high];
13     int i = (low - 1);
14
15     for (int j = low; j <= high - 1; j++) {
16         // Change Logic to greater than for descending
17         if (arr[j] > pivot) {
18             i++;
19             swap(&arr[i], &arr[j]);
20         }
21     }
22     swap(&arr[i + 1], &arr[high]);
23     return (i + 1);
24 }
25
26 void quickSortDescending(long long arr[], int low, int high) {
27     if (low < high) {
28         int pi = partitionDescending(arr, low, high);
29         quickSortDescending(arr, low, pi - 1);
30         quickSortDescending(arr, pi + 1, high);
31     }
32 }
33
34 int main() {
35     int N;
36     scanf("%d", &N);
```



main.c

```
34 int main() {
35     int N;
36     scanf("%d", &N);
37
38     long long *sales = (long long *)malloc(N * sizeof(long long));
39     for(int i = 0; i < N; i++) {
40         scanf("%lld", &sales[i]);
41     }
42
43     // Sort in Descending order
44     quickSortDescending(sales, 0, N - 1);
45
46     // 1. Print sorted sales
47     for(int i = 0; i < N; i++) {
48         printf("%lld ", sales[i]);
49     }
50     printf("\n");
51
52     // 2. Print Top 3
53     printf("Top 3: ");
54     for(int i = 0; i < 3 && i < N; i++) {
55         printf("%lld ", sales[i]);
56     }
57     printf("\n");
58
59     // 3. Print Average of top 3
60     long long sum = 0;
61     int count = (N >= 3) ? 3 : N;
62     for(int i = 0; i < count; i++) {
63         sum += sales[i];
64     }
65     printf("Average: %lld\n", sum / count);
66
67     free(sales);
68     return 0;
69 }
```

```

40     scanf("%lld", &sales[i]);
41 }
42
43 // Sort in Descending order
44 quickSortDescending(sales, 0, N - 1);
45
46 // 1. Print sorted sales
47 for(int i = 0; i < N; i++) {
48     printf("%lld ", sales[i]);
49 }
50 printf("\n");
51
52 // 2. Print Top 3
53 printf("Top 3: ");
54 for(int i = 0; i < 3 && i < N; i++) {
55     printf("%lld ", sales[i]);
56 }
57 printf("\n");
58
59 // 3. Print Average of top 3
60 long long sum = 0;
61 int count = (N >= 3) ? 3 : N;

```

input

```

6 1200 5000 3000 2500 7000 4500
7000 5000 4500 3000 2500 1200
Top 3: 7000 5000 4500
Average: 5500

```

```

...Program finished with exit code 0
Press ENTER to exit console.

```